

Evaluating Modern Web Performance Optimization Techniques in a React-Based Educational Web Application: An Experimental Study

Independent Research Project | EduFlow Experimental Web Application

RESEARCH PAPER

Researcher: Ahmed Mohamed Saleh

Abstract

This study experimentally evaluates selected modern web-performance optimization techniques in a React-based educational web application. EduFlow was implemented in six configurations: V0 Baseline, V1 Image Optimization, V2 Lazy Loading, V3 Code Splitting, V4 Delivery Optimization, and V5 Combined Optimization. Each configuration was evaluated using five Lighthouse navigation runs under the same emulated mobile environment. Mean and sample standard deviation were calculated for Performance Score, First Contentful Paint (FCP), Largest Contentful Paint (LCP), Total Blocking Time (TBT), Cumulative Layout Shift (CLS), and Speed Index. Under the tested conditions, V2 produced the lowest mean FCP, LCP, TBT, and Speed Index. V3–V5 did not improve every metric, illustrating that optimization techniques can interact with application structure and measurement conditions. The findings are limited to the tested application and environment.

1. Introduction

Web performance is influenced by resource size, image delivery, JavaScript execution, rendering order, network conditions, and visual stability. Modern client-side applications therefore require empirical validation rather than assuming that every recommended optimization will improve every workload. This study compares a controlled baseline with incremental optimization configurations on the same React application and evaluates several complementary performance metrics.

2. Problem Statement

Developers commonly adopt image optimization, lazy loading, code splitting, and delivery optimizations as established practices. However, the magnitude and direction of their effect depend on workload characteristics and execution conditions. The research problem addressed here is how these techniques behave when introduced systematically into one controlled React workload.

3. Research Gap

Existing technical guidance explains how individual techniques can reduce resource cost or startup work, while academic work has examined broader web-performance optimization strategies. What remains useful at the application level is a reproducible, workload-specific comparison in which multiple techniques are implemented on the same application and measured repeatedly. This study addresses that practical gap through six versioned EduFlow configurations and 30 recorded Lighthouse runs.

4. Related Work / Literature Review

Prior work has treated web performance optimization as a combination of resource and asset strategies, including caching and prefetching approaches. IEEE research on smart resource and asset optimization demonstrates that coordinated resource strategies can affect web application performance.

Current web-platform guidance also emphasizes that image optimization can reduce transfer costs, while lazy loading can defer off-screen resources and reduce competition for critical resources. At the same time, Google's performance guidance cautions that applying lazy loading to above-the-fold or LCP images can delay their discovery and harm LCP.

This caution is particularly relevant to the present experiment: V2 performed best overall among the tested versions, but the result should not be generalized into a claim that lazy loading is universally superior. Published experimental guidance explicitly shows that the effect of lazy loading can vary with page structure and which images are deferred.

Code splitting is similarly motivated by reducing the JavaScript payload required during startup. Web.dev describes code splitting as a way to reduce initial JavaScript and main-thread work, while noting that application architecture influences its effect.

5. Technical Background

Image optimization. Reducing image dimensions and compression can lower transferred bytes. The benefit depends on whether image transfer is a material part of the critical loading path.

Lazy loading. The browser can defer off-screen images until they approach the viewport. This can reduce bandwidth contention, but above-the-fold lazy loading can delay LCP.

Code splitting. Route-level splitting can reduce the JavaScript required at startup and may reduce parsing, compilation, and execution work. Its effect depends on the application's rendering architecture.

Delivery optimization. Compression, caching, and connection/resource hints can influence transfer efficiency and resource discovery; the measurable effect depends on the actual deployment and network path.

Performance metrics. FCP measures early visible rendering, LCP measures the largest relevant content element, TBT summarizes blocking during loading, CLS measures unexpected layout movement, and Speed Index summarizes visual completion during loading.

6. Contribution of This Study

The contribution is experimental rather than algorithmic. The study provides a realistic React educational workload, six versioned optimization configurations, repeated Lighthouse measurements, mean and sample-standard-deviation analysis, and an explicit interpretation of both improvements and regressions. It also documents the environment sufficiently to support reproduction of the laboratory procedure.

7. System Design and Experimental Versions

V0 — Baseline: reference implementation without the study's optimization changes.

V1 — Image Optimization: reduced image dimensions/quality parameters while preserving application behavior.

V2 — Lazy Loading: defers non-critical image/resource loading.

V3 — Code Splitting: route-level lazy loading using React.lazy and Suspense.

V4 — Delivery Optimization: production-oriented delivery configuration including compression/cache-oriented serving and connection hints.

V5 — Combined: combines the preceding techniques.

8. Experimental Methodology

Each version was built as a production application and tested with Lighthouse navigation mode. Five runs were collected per version, yielding 30 result files. For every metric, the arithmetic mean and sample standard deviation were calculated from the five observations.

Environment: Windows NT 10.0, Win64/x64; HeadlessChrome 155.0.0.0; Lighthouse 13.5.0; mobile emulation; 412 × 823 viewport; device scale factor 1.75; CPU slowdown 4×; RTT 150 ms; throughput 1638.4 Kbps; download throughput 1474.56 Kbps; upload 675 Kbps; request latency 562.5 ms; navigation gather mode. The extracted configuration did not establish an independent cache-state condition, so cache state is not treated as a controlled variable.

9. Results

Metric	V0 Mean ± SD	V1 Mean ± SD	V2 Mean ± SD	V3 Mean ± SD	V4 Mean ± SD	V5 Mean ± SD
Performance	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00	98.00 ± 0.00	98.00 ± 0.00	98.00 ± 0.00
FCP (ms)	1216.29 ± 28.23	1214.34 ± 20.90	1204.12 ± 1.44	1299.52 ± 19.20	1289.47 ± 46.47	1306.99 ± 70.72
LCP (ms)	1336.29 ± 38.88	1334.34 ± 46.23	1324.12 ± 67.49	1544.89 ± 38.12	1504.99 ± 81.87	1525.84 ± 62.83
TBT (ms)	8.00 ± 4.30	14.70 ± 12.61	6.70 ± 0.97	5.30 ± 3.23	5.90 ± 5.10	8.70 ± 4.63
CLS	0.0000 ± 0.0000	0.0000 ± 0.0000	0.0000 ± 0.0000	0.0826 ± 0.0000	0.0826 ± 0.0000	0.0826 ± 0.0000

Metric	V0 Mean ± SD	V1 Mean ± SD	V2 Mean ± SD	V3 Mean ± SD	V4 Mean ± SD	V5 Mean ± SD
Speed Index (ms)	1216.29 ± 28.23	1214.34 ± 20.90	1204.12 ± 1.44	1334.03 ± 58.02	1289.47 ± 46.47	1355.55 ± 168.96

10. Visual Analysis

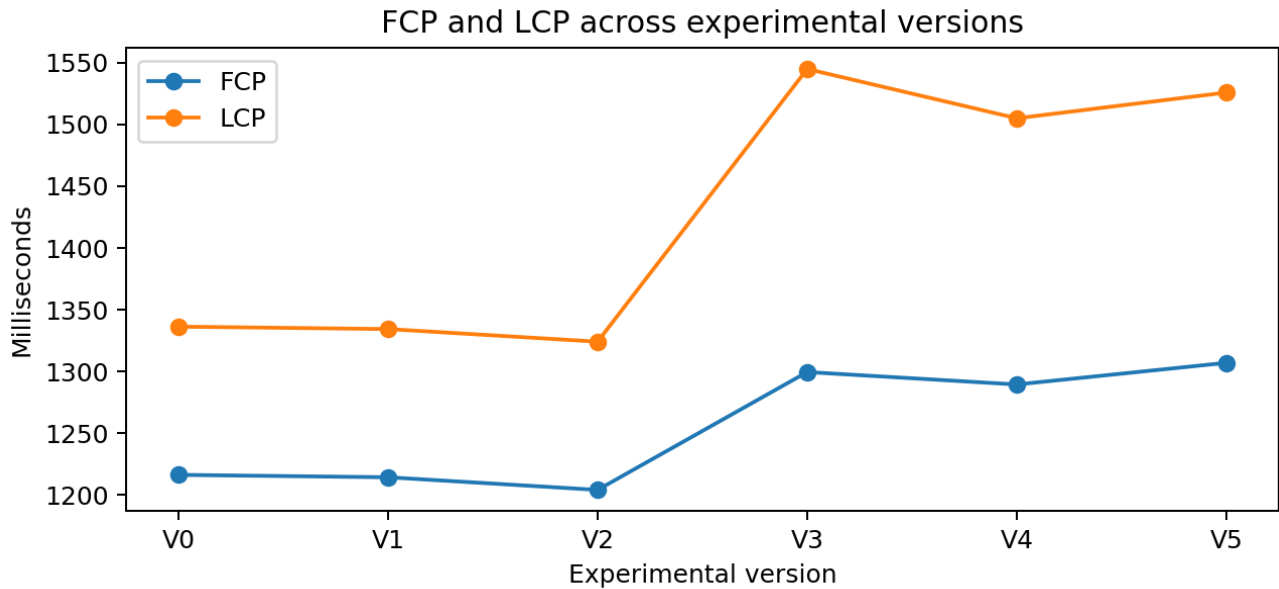


Figure 1. FCP and LCP. V2 has the lowest mean values among the six configurations, while V3–V5 show higher mean LCP than V0. This pattern indicates that the later optimizations did not translate into faster loading of the principal content under the tested conditions.

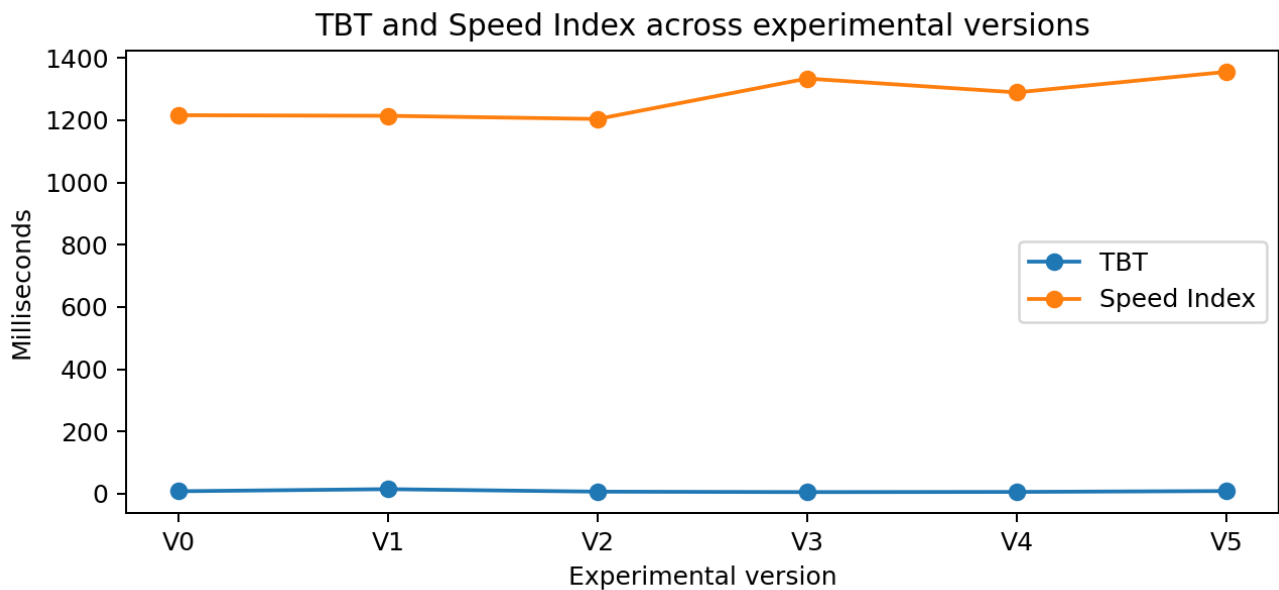


Figure 2. TBT and Speed Index. V2 has the lowest mean TBT and Speed Index. V3 records the lowest TBT, but this does not correspond to the best FCP/LCP or overall Performance Score, illustrating why multiple metrics are needed.

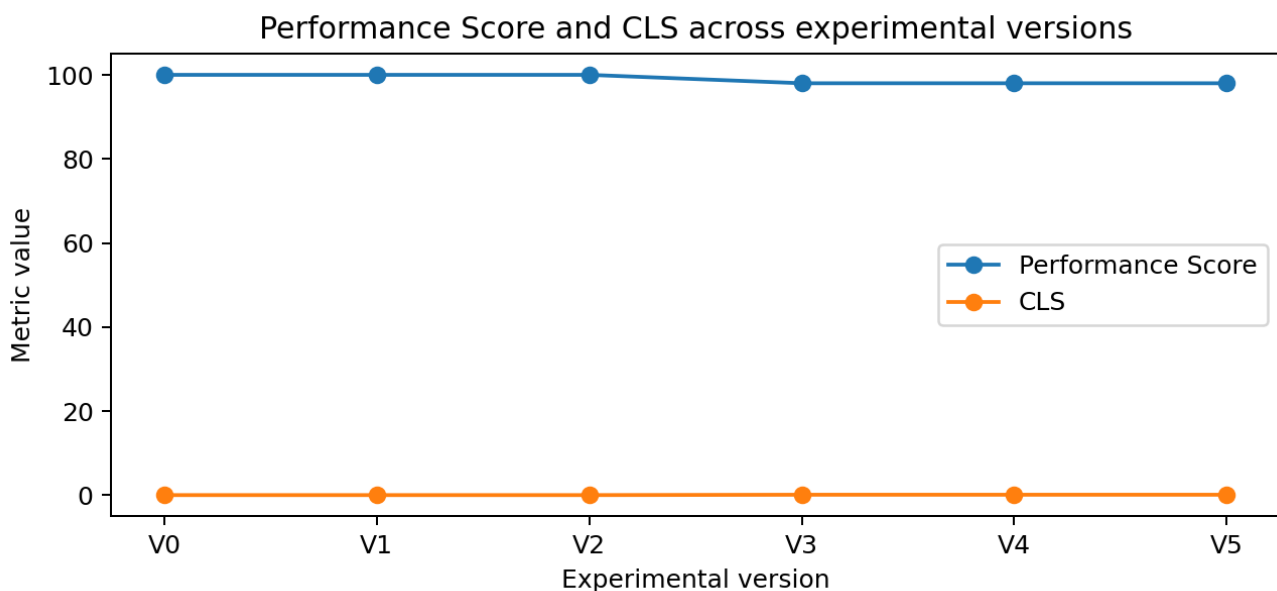


Figure 3. Performance Score and CLS. V0–V2 maintain a Performance Score of 100 and CLS of 0, whereas V3–V5 show a score of 98 and CLS of 0.0826. The simultaneous change suggests that later configurations altered visual stability and/or other score components in this workload.

11. Discussion

Relative to V0, V2 reduced mean FCP from 1216.29 ms to 1204.12 ms ($\approx 1.0\%$), LCP from 1336.29 ms to 1324.12 ms ($\approx 0.9\%$), TBT from 8.00 ms to 6.70 ms ($\approx 16.3\%$), and Speed Index from 1216.29 ms to 1204.12 ms ($\approx 1.0\%$). These changes are workload-specific and should not be interpreted as proof that lazy loading is generally the best optimization.

V1 produced only small changes in the measured loading metrics. This does not demonstrate that image optimization lacks value; it indicates that the particular image changes and workload used here did not substantially alter the selected Lighthouse metrics.

V3–V5 require cautious interpretation. Their FCP and LCP means were higher than V0, and CLS increased from 0 to 0.0826. One plausible explanation is that route splitting and delivery changes modified resource discovery, request sequencing, or layout behavior. However, the experiment does not isolate the exact causal mechanism for the CLS increase, so this interpretation remains a hypothesis rather than a demonstrated cause.

The Performance Score decreased from 100 in V0–V2 to 98 in V3–V5. Because the score is composite, the individual metrics provide more useful diagnostic information than the score alone.

12. Threats to Validity and Limitations

The study uses one application, one host environment, one emulated mobile profile, and five runs per configuration. Lighthouse results may vary with runtime conditions. The experiment therefore supports conclusions about the tested EduFlow implementation rather than all React applications. Cache state was not independently established. No field data, user study, or cross-browser comparison was included.

13. Reproducibility

The project contains the application, versioned experiment configurations, Lighthouse automation, and result-processing scripts. Reproduction consists of building each version, running five Lighthouse navigation measurements under the documented configuration, and calculating the reported statistics from the recorded JSON files.

14. Conclusion

This study provides a controlled comparison of common web-performance optimization techniques in a React-based educational application. Under the specified laboratory conditions, V2 produced the lowest mean FCP, LCP, TBT, and Speed Index. V3–V5 did not improve every metric, reinforcing the importance of workload-specific measurement and multi-metric analysis. The study's contribution is therefore a transparent experimental framework and evidence-based comparison, not a universal ranking of optimization techniques.

References

- [1] IEEE, "Web performance optimization through smart resource and asset optimizations," in Proc. INDIACom, [Online]. Available: IEEE Xplore, document 7724271.
- [2] Google, "Key performance issues," web.dev.
- [3] Google, "Lazy load images and iframe elements," web.dev.
- [4] F. Arntz and R. Viscomi, "The performance effects of too much lazy loading," web.dev.
- [5] Google, "Optimize Largest Contentful Paint," web.dev.
- [6] Google, "Reduce JavaScript payloads with code splitting," web.dev.
- [7] Google, "Eliminating unnecessary downloads," web.dev.
- [8] Google, "Web performance made easy," web.dev.
- [9] Google, "Image performance," web.dev.
- [10] IEEE, "A Dynamic Approach of Web Pages' Rendering based on Network Speed," doi: 10.1109/GCON65540.2025.11173364.
- [11] IEEE, "Analyzing Dead Code Elimination Techniques for Green and Efficient Web Applications," doi: 10.1109/ICCCA66364.2025.11325681.

Academic status note: This document is an Independent Research Project prepared in support of an Open Doors Scholarship application. It does not claim to be an awarded Master's thesis, an institutional Master's research project, or a published paper.